

Dataflow Through the Network

The dataflow in the architecture follows a structured top-down and bottom-up autoencoder progression, enabling hierarchical feature extraction, contextual integration, and precise voxel-wise reconstruction. The input data is fed to the encoder side and the segmented output is received from the decoder side.

We can understand the dataflow through the architecture step by step starting from the raw BraTS Data to the final segmentation map.

1. Input Preprocessing and Normalization

The .nii files that represent the data are passed through a custom data generator that stacks 3 out of the 4 'channels' belonging to an input data into a numpy array of size $128 \times 128 \times 128 \times 3$

Each subject is represented as a pre-aligned multi-modal MRI volume:

$$X_0 = \{X^{(1)} X^{(2)} X^{(3)} X^{(4)}\} - \{X^{(1)}\} \in \mathbb{R}^{H \times W \times D \times 3}$$

Where $X^{(1)}, X^{(2)}, X^{(3)}, X^{(4)}$ correspond to the T1, T1CE, T2, and FLAIR modalities respectively, and $H = W = D = 128$

Before entering the network, each modality is min-max normalized independently to $[0, 1]$. The three channels are stacked to form the initial 3D input tensor in the form of a NumPy array converted from the .nii file format. The height, width and depth of 128 is achieved by

2. Encoder (Contracting Path)

At each encoder stage $\ell \in \{1, 2, 3, 4\}$, the network performs three sequential operations:

- **Feature Extraction (Dense Inception Residual Convolution):**
Each stage applies a DRIC block, combining multiple convolutional branches with different receptive fields
The block output is: $F_\ell = \mathcal{DRIC}_\ell(X_{\ell-1})$
- **Downsampling:**
The feature map is halved in each spatial dimension using 3D max pooling:
 $X_\ell = \text{MaxPool3D}(F_\ell, p = 2)$. Here p represents the scaling factor.
- **Skip Connection Storage:**
Each pre-pooled feature map F_ℓ is preserved for later fusion with the decoder at the corresponding level.

3. Multiscale Input

The input in the form of a numpy array is downsampled through a simple average pooling function that converts the primary input of size $128^3 \times 3$ into multiple scaled down inputs of $128^3 \times 3$, $64^3 \times 3$, and $32^3 \times 3$; that is then fed to the encoder at different stages.

The original input X_0 (of size $128^3 \times 3$) is successively downsampled:

$\tilde{X}_\ell = \text{Downsample}(X_0, \text{scale} = 2^\ell)$, ℓ represents the encoder stage.

and concatenated with X_ℓ before entering the next DRIC block:

$$\hat{X}_\ell = [X_{\ell-1} \parallel \tilde{X}_\ell].$$

4. Bottleneck (Bridge)

The lowest-level encoder output passes through a bottleneck DRIC block,

$$B = \mathcal{DRIC}_{\text{bottleneck}}(X_4).$$

5. Decoder (Expanding Path)

The decoder path performs progressive upsampling and reconstruction to return the segmentation to full spatial resolution.

At each decoder stage $m \in \{4, 3, 2, 1\}$:

1. Upsampling:

The feature map is expanded using transposed 3D convolutions:

$U_m = \text{ConvTranspose3D}(B_m, k = 2, s = 2)$, where k and s denote kernel size and stride respectively.

2. Attention-Gated Skip Connection:

Before concatenating with its encoder counterpart F_m , an **attention gate** filters the encoder feature map based on relevance to the decoder's context.

Let the decoder's gating signal be g_m and the encoder feature be F_m ; the attention coefficient is computed as:

$$\alpha_m = \sigma(W_g * g_m + W_x * F_m + b),$$

and the refined feature map is: $\tilde{F}_m = \alpha_m \odot F_m$, where $\odot$ denotes element-wise multiplication.

This suppresses irrelevant background and enhances focus on tumor boundaries and salient regions.

3. Fusion and Convolution:

The attention-refined skip feature $\tilde{F}_m$ is concatenated with U_m :

$$C_m = [U_m \parallel \tilde{F}_m],$$

which is then processed by another DRIC block: $B_{m-1} = \mathcal{DRIC}_m(C_m)$.

6. Output Layer

After the final decoder stage, a $1 \times 1 \times 1$ convolution projects the feature volume into the four output classes: $Y = \text{Softmax}(W_{\text{out}} * B_0 + b_{\text{out}})$,

where $Y \in [0,1]^{H \times W \times D \times 4}$ represents per-voxel class probabilities for background, NCR/NET, ED, and ET.